TP2 — Analyse syntaxique de MICRO

`tp2.c` est un programme C autonome : il lit le fichier de tokens produit par le TP1, puis applique un analyseur syntaxique descendant récursif correspondant aux photos, pages 32–35.

Sources et lecture du document

Les **pages 32–35 des photos** donnent les règles et les fonctions de l'exercice. Les numéros indiqués comme **pages du cours** ci-dessous désignent le PDF [MonCourscompilationILIS (1).pdf](<../../cours/MonCourscompilationILIS (1).pdf>). Ses numéros affichés correspondent aux positions dans le PDF. Il faut donc distinguer les deux documents, même lorsqu'ils ont un numéro en commun.

Pages du cours	Notions utilisées
20–21	Analyse syntaxique : structures grammaticales et arbre de dérivation
22	Analyse sémantique et contrôles de types
23	Génération de code, phase distincte de la reconnaissance
30	Table des symboles
34	Générateurs de scanners et de parsers : Lex/Flex, Bison/Yacc
38	Interface entre analyse lexicale et syntaxique ; numéros de ligne
39–40	Tokens, lexèmes, modèles et nombres réels
100	Appels successifs entre scanner et analyseur syntaxique
107	Erreurs lexicales et possibilités de récupération

Le cours donne les principes généraux ; les photos donnent la grammaire MICRO à implémenter. Les choix techniques C et l'arrêt à la première erreur sont des choix de notre programme, pas des obligations attribuées au cours.

Chaînage TP1 → TP2

Les deux TPs forment une petite chaîne de compilation :

```
exemple.txt --TP1--> unites_lexicales.txt --TP2--> programme valide ou erreur
```

Le TP1 lit le programme MICRO caractère par caractère et écrit un token par ligne. Le fichier commence par deux lignes d'en-tête, puis chaque ligne contient la ligne d'origine, le nom du token et son lexème :

```
LIGNE    UNITE           LEXEME
-----    -----           ------
1        BEGIN           BEGIN
2        READ            READ
2        LPAREN          (
2        ID              nombre
```

Le TP2 ignore les en-têtes, lit les colonnes et reconstruit le token courant. Il ne relit donc pas le texte source et ne fait pas une deuxième analyse lexicale. Il vérifie l'ordre des tokens avec la grammaire syntaxique. La colonne `LIGNE` est conservée pour localiser les erreurs.

Compiler et exécuter

Depuis la racine du dépôt, dans PowerShell :

```
gcc -std=c11 -Wall -Wextra -Wpedantic .\TP1\tp1.c -o .\TP1\tp1.exe
gcc -std=c11 -Wall -Wextra -Wpedantic .\TP2\tp2.c -o .\TP2\tp2.exe
.\TP1\tp1.exe .\TP1\exemple.txt .\TP1\unites_lexicales.txt
.\TP2\tp2.exe .\TP1\unites_lexicales.txt
```

Le premier programme crée `TP1/unites_lexicales.txt`. Le second reçoit ce fichier comme entrée et affiche le résultat dans le terminal. Le code de retour est 0 en cas de réussite et non nul en cas d'erreur. L'analyse s'arrête à la première erreur détectée.

Pour tester le fichier fourni avec le TP2, il faut d'abord le faire passer par le TP1 :

```
.\TP1\tp1.exe .\TP2\exemple.txt .\TP2\unites_lexicales.txt
.\TP2\tp2.exe .\TP2\unites_lexicales.txt
```

Lancer les deux TPs avec le programme C++

Le fichier `run_pipeline.cpp`, placé à la racine, automatise les quatre étapes : compiler le TP1, compiler le TP2, générer les tokens, puis vérifier la syntaxe. Depuis la racine du dépôt :

```
g++ -std=c++17 -Wall -Wextra -Wpedantic run_pipeline.cpp -o run_pipeline
./run_pipeline
```

Par défaut, il utilise `TP1/exemple.txt` et écrit les tokens dans `TP1/unites_lexicales.txt`. On peut fournir un autre fichier source et un autre fichier de sortie :

```
./run_pipeline TP2/exemple.txt /tmp/tokens_tp2.txt
```

Dans cet exemple, le TP1 réussit l'analyse lexicale, puis le TP2 signale les erreurs syntaxiques présentes dans `TP2/exemple.txt`.

Grammaire reconnue

Les accolades signifient une répétition zéro ou plusieurs fois.

```
system_goal → program SCANEOF
program     → BEGIN inst_list END
inst_list   → inst { inst }
inst        → ID := expression ;
            | READ ( id_list ) ;
            | WRITE ( expr_list ) ;
id_list     → ID { , ID }
expr_list   → expression { , expression }
expression  → prim { add_op prim }
add_op      → + | -
prim        → ( expression ) | ID | INTLITERAL | REALLITERAL
```

`REALLITERAL` prolonge l'extension du TP1 : les photos ne prévoyaient que `INTLITERAL`. Les mots réservés sont en majuscules. Au moins une instruction est obligatoire ; les listes READ et WRITE ne peuvent pas être vides. Les signes unaires, la multiplication, la division et les exposants ne sont pas prévus par cette version. `x := -2.5;` est donc refusé, mais `x := 0 - 2.5;` est accepté.

Fonctions et corrections

puis termine le programme.

`t` et `tok` sur les photos.

programme après avoir supprimé silencieusement un caractère incorrect.

- `next_token()` consulte le token mémorisé sans consommer l'entrée.
- `match()` vérifie le token attendu puis appelle le scanner pour avancer.
- Chaque fonction grammaticale reconnaît son non-terminal.
- `syntax_error()` affiche la ligne, ce qui était attendu et le token rencontré,
- `system_goal()` vérifie la fin du fichier : aucun token ne peut suivre END.
- `add_op()` utilise une seule variable `tok`, corrigeant l'incohérence entre
- Les noms des mots réservés sont adaptés à ceux du TP1 (`BEGIN_TOKEN`, etc.).
- Le scanner intégré s'arrête sur erreur lexicale, afin de ne pas accepter un

L'analyseur vérifie la structure des instructions ; il ne calcule pas les expressions, n'exécute pas READ/WRITE et ne vérifie pas les déclarations.

Vérification

Compilation vérifiée avec `-std=c11 -Wall -Wextra -Wpedantic -Werror`. Quinze cas ont été vérifiés : listes et parenthèses valides, formes décimales, commentaire sans retour à la ligne final, point-virgule manquant, virgule finale dans READ, WRITE vide, opérande absent, parenthèse manquante, texte après END, programme sans instruction, signe unaire, caractère illégal, fin prématurée, notation scientifique non admise, point seul et fichier vide.

Explication détaillée, étape par étape

1. Définir précisément le rôle du TP2

Le TP1 répond à « quelles unités lexicales trouve-t-on ? ». Le TP2 répond à « sont-elles assemblées conformément à la grammaire MICRO ? ».

Par exemple, `x := 10 + ;` contient uniquement des tokens reconnus, mais ne respecte pas la règle d'expression : un primaire manque après +. Le scanner peut reconnaître tous ces tokens ; le parser doit refuser leur ordre.

Choix : construire un reconnaisseur syntaxique. Nous ne calculons pas les expressions et ne construisons pas d'arbre enregistré en mémoire. La structure est vérifiée implicitement par les appels de fonctions.

Cours : pages 20–21, regroupement des tokens en structures grammaticales et arbre de dérivation. Les contrôles de types relèvent de la **page 22** ; la production de code relève de la **page 23** et dépasse ce TP.

2. Lire la sortie du TP1

Le parser appelle `scanner()` directement, mais ce scanner lit maintenant une ligne du fichier `unites_lexicales.txt` à chaque appel. Il ne reconnaît plus les caractères du programme source : cette responsabilité appartient au TP1. L'interface reste la même pour le parser : le token retourné, `token_buffer` pour le lexème et `token_line` pour la ligne d'origine.

Choix : garder un fichier C autonome, mais séparer clairement les deux étapes. Le TP1 produit le fichier intermédiaire ; le TP2 le consomme. Ainsi, on peut inspecter les tokens avant de lancer l'analyse syntaxique et comprendre où se trouve une erreur.

Le lecteur vérifie que chaque nom d'unité (`BEGIN`, `ID`, `INTLITERAL`, etc.) est connu. Une ligne mal formée ou un token inconnu provoque une erreur immédiate au lieu d'être ignoré.

Cours : pages 38 et 100, le scanner fournit les unités à l'analyseur syntaxique. **Pages 39–40**, distinction entre catégorie et lexème.

3. Traduire la grammaire en fonctions C

Chaque non-terminal devient une fonction : `program()`, `inst()`, `prim()`, etc. Les terminaux sont vérifiés par `match()` ; une séquence devient une suite d'appels ; une alternative devient un `switch` ; une répétition devient une boucle.

```
Règle : program → BEGIN inst_list END
Code  : match(BEGIN_TOKEN); inst_list(); match(END_TOKEN);
```

Cette méthode est une **analyse descendante récursive** : on part de la règle globale et on descend vers les constituants. La récursion intervient notamment entre `expression()` et `prim()` pour les parenthèses imbriquées.

Choix : suivre les fonctions des photos plutôt que générer le parser. Le token courant suffit à choisir les alternatives de cette grammaire : `ID`, `READ` et `WRITE` distinguent les formes d'instructions.

Cours : pages 20–21, grammaires et structure hiérarchique ; **page 34**, Bison/Yacc comme autre moyen de produire un analyseur. L'organisation concrète en fonctions vient des **photos, pages 32–35**.

4. Mémoriser un token d'anticipation : next_token()

```
static token current_token;

static token next_token(void)
{
    return current_token;
}
```

`next_token()` consulte sans avancer. Plusieurs fonctions doivent pouvoir observer le même token. Pour `READ(x);`, `inst_list()` vérifie qu'une instruction commence, `inst()` choisit READ, puis `match(READ_TOKEN)` vérifie cette même unité.

Choix : conserver un seul token d'anticipation. Appeler le scanner à chaque consultation ferait perdre des tokens et casserait le fonctionnement des photos. La lecture initiale est effectuée une seule fois dans `main()`.

Cours : pages 38 et 100, circulation des tokens entre scanner et parser. Le mécanisme de consultation est notre implémentation de cette interface, en réponse aux appels à `next_token()` présents dans les photos.

5. Vérifier et consommer : match()

```
static void match(token expected)
{
    if (next_token() != expected) {
        char message[128];
        if (expected == SCANEOF)
            snprintf(message, sizeof message, "fin du fichier attendue");
        else
            snprintf(message, sizeof message, "%s attendu", token_description(expected));
        syntax_error(message);
    }

    if (expected != SCANEOF)
        current_token = scanner();
}
```

Si le token convient, `match()` avance exactement une fois. Sinon, il signale l'erreur avant d'avancer. Ainsi, le lexème et la ligne du token incorrect sont encore disponibles pour le diagnostic.

Pour SCANEOF, nous vérifions simplement la fin : il n'est pas nécessaire d'effectuer une nouvelle lecture après ce marqueur.

Cours : page 20, vérification des tokens attendus dans une construction ; **page 100**, demande de l'unité suivante après traitement de l'unité courante.

6. Reconnaître tout le fichier : system_goal() et program()

`program()` impose BEGIN, une liste d'instructions, puis END. `system_goal()` impose ensuite SCANEOF.

Choix : vérifier la totalité du fichier. Sans le dernier contrôle, le parser pourrait accepter un préfixe valide et oublier du texte après END. Les blancs et commentaires après END sont permis, puisqu'ils sont éliminés par le scanner avant le contrôle de fin.

```
BEGIN WRITE(1); END             → accepté
BEGIN WRITE(1); END WRITE(2);   → SCANEOF attendu, WRITE rencontré
```

Photos : page 32. Cours : pages 20–21, respect de la structure globale ; **page 100**, traitement du programme source jusqu'à sa fin.

7. Reconnaître au moins une instruction : inst_list()

```
inst();
while (next_token() == ID || next_token() == READ_TOKEN ||
       next_token() == WRITE_TOKEN)
    inst();
```

Le premier appel est obligatoire. La boucle reconnaît les instructions supplémentaires, dont les premiers tokens possibles sont ID, READ et WRITE. Elle laisse le token qui ne commence pas une instruction au niveau appelant. `program()` vérifiera alors qu'il s'agit de END.

Choix : respecter `inst_list → inst { inst }`. Les accolades signifient zéro ou plusieurs répétitions après la première instruction. BEGIN END est donc invalide, conformément aux photos.

Photos : page 33. Cours : page 20, construction des suites d'instructions selon des règles grammaticales.

8. Choisir la bonne instruction : inst()

Le `switch` utilise uniquement le token courant :

Premier token	Forme à vérifier	Raisonnement
`ID`	`ID := expression ;`	Un identificateur initial introduit une affectation
`READ_TOKEN`	`READ ( id_list ) ;`	READ reçoit des identificateurs
`WRITE_TOKEN`	`WRITE ( expr_list ) ;`	WRITE reçoit des expressions
Autre	Diagnostic	Aucun début d'instruction prévu ne correspond

Chaque branche consomme tous les terminaux requis et appelle les fonctions des constituants. Le point-virgule est obligatoire pour les trois formes.

Choix : ne pas ajouter d'autres instructions. READ et WRITE sont reconnus comme constructions grammaticales ; ils ne réalisent pas d'entrée/sortie MICRO.

Photos : page 33. Cours : pages 20–21, structures des instructions et expressions. **Page 22**, les contrôles de sens sont une autre phase.

9. Reconnaître les listes : id_list() et expr_list()

`id_list()` commence par `match(ID)`, puis répète `COMMA` et `ID`. `expr_list()` commence par `expression()`, puis répète `COMMA` et une expression.

Choix : imposer au moins un élément et un élément après chaque virgule. Ainsi, `READ()` et `WRITE()` sont refusés, tout comme `READ(x,)`. `READ(2)` est refusé : un nombre n'est pas un identificateur. `WRITE(x + 2, 3.5)` est accepté : WRITE peut contenir des expressions.

Les listes ne consomment pas la parenthèse fermante : `inst()` la vérifie. Chaque fonction a ainsi la responsabilité exacte de son non-terminal.

Photos : pages 33–34. Cours : page 20, vérification des constituants et de leur ordre dans une construction grammaticale.

10. Reconnaître les expressions : expression() et add_op()

`expression()` commence par `prim()`, puis répète un opérateur additif suivi d'un nouveau primaire :

```
prim("identificateur, nombre ou expression entre parentheses attendu");
while (next_token() == PLUSOP || next_token() == MINUSOP) {
    char message[128];
    token op = next_token();
    snprintf(message, sizeof message, "operande attendu apres %s",
             token_description(op));
    add_op();
    prim(message); /* Contexte : operande attendu apres '+' ou '-'. */
}
```

Cela accepte `x + 2 - y`, mais refuse `x + ;` puisqu'un opérateur doit être suivi d'un primaire. `add_op()` accepte uniquement PLUSOP ou MINUSOP.

Correction des photos : la variable déclarée dans `add_op()` est `t`, mais la condition utilise `tok`. Notre fonction utilise `tok` partout.

Choix : conserver uniquement les opérateurs prévus. Nous n'ajoutons ni `*` ni `/`. La boucle reconnaît une chaîne additive ; elle ne calcule rien et ne construit pas d'arbre matérialisant l'associativité des

opérations.

Photos : page 34. Cours : pages 20–21, construction hiérarchique des expressions et arbre de dérivation.

11. Reconnaître les primaires et les parenthèses : prim()

Les alternatives sont un identificateur, un entier, un réel ou une expression parenthésée. Dans ce dernier cas :

```
match(LPAREN);
expression();
match(RPAREN);
```

L'appel récursif autorise des parenthèses imbriquées comme `((x + 2) - y)`. Chaque ouverture doit avoir sa fermeture, vérifiée au retour de l'expression. Une paire vide `()` est invalide, puisque l'expression exige un premier primaire.

Ajout : `case REALLITERAL` prolonge le TP1. Sans lui, le scanner reconnaîtrait `10.32`, mais le parser le refuserait, car les photos n'acceptent que les entiers.

Choix : ne pas ajouter de signe unaire. Le scanner retourne MINUSOP pour `-`, et la grammaire ne permet ce token qu'entre deux primaires. `-2.5` est donc refusé comme expression, tandis que `0 - 2.5` est accepté.

Photos : page 35. Cours : pages 20–21, structure imbriquée ; **page 40**, modèles des réels. Le cours présente aussi des exposants et des nombres signés : notre version ne reprend pas toutes ces formes.

12. Arrêter à la première erreur et expliquer le diagnostic

`syntax_error()` affiche le fichier, la ligne, une attente formulée en français et le lexème rencontré, ou la fin du fichier. `token_description()` traduit les catégories internes en descriptions lisibles (`';'`, un `identificateur`, etc.). Après un opérateur, `expression()` transmet à `prim()` un contexte précis : `operande attendu apres '-'` ou `operande attendu apres '+'`. La fonction ferme le fichier et appelle `exit(EXIT_FAILURE)`.

```
exemple.txt:3: erreur syntaxique : ';' attendu ; 'END' rencontre
```

La ligne indiquée est celle du **token rencontré**, donc l'endroit où le problème est détecté. Dans cet exemple, le point-virgule peut manquer sur la ligne précédente. Un diagnostic ne désigne pas toujours le caractère à modifier.

Choix : s'arrêter au premier problème, conformément au comportement retenu pour ce TP. Continuer nécessiterait une récupération et une resynchronisation, par exemple à un séparateur, afin d'éviter une cascade de faux diagnostics. Les photos ne définissent pas cette stratégie : nous l'avons explicitement choisie.

Le scanner intégré s'arrête aussi sur une erreur lexicale. Contrairement au TP1, il ne se contente pas de compter puis ignorer les caractères incorrects. Sinon `WRITE(1@);` pourrait devenir `WRITE(1);` pour le parser et être accepté à tort après l'erreur. Le diagnostic lexical termine donc aussi le TP2.

Cours : page 20, exemple d'un token attendu manquant ; **page 38**, suivi des lignes. **Page 107** traite la récupération lexicale : elle fournit un contexte, mais ne prescrit pas notre politique d'arrêt syntaxique.

13. Initialiser et terminer proprement dans main()

Le programme exige exactement un argument : le nom du fichier MICRO. Il vérifie `fopen()`, initialise `current_token = scanner()`, puis appelle `system_goal()`.

Après une reconnaissance réussie, `ferror()` vérifie qu'une erreur de lecture n'a pas été confondue avec une fin normale. Le fichier est fermé, puis le programme affiche le succès et retourne `EXIT_SUCCESS`. Les erreurs retournent `EXIT_FAILURE`.

Choix : afficher le résultat dans le terminal, sans fichier de rapport. Les photos utilisent des fonctions `void` et n'imposent aucun fichier résultat. Le message de succès signifie « conforme à cette grammaire », sans garantie sur les types, les déclarations ou le résultat d'une éventuelle exécution.

Cours : pages 38 et 100, chaîne de traitement scanner/parser ; **pages 22–23**, distinction avec la sémantique et la génération de code. La gestion des arguments, des fichiers et des codes de retour est un choix d'implémentation C.

14. Vérifier la reconnaissance et les refus

La compilation avec les avertissements activés contrôle les problèmes C. Elle ne prouve pas que le parser respecte la grammaire : nous vérifions aussi des entrées valides et invalides, avec leur code de retour et leur diagnostic.

Cas	Résultat attendu	Règle ou choix vérifié
`READ(x,y);` dans un programme	Succès	Liste d'identificateurs
`WRITE((x+2),.5);` dans un programme	Succès	Parenthèses, listes et réels
Commentaire après END sans retour final	Succès	Fin de fichier dans le scanner
`BEGIN END`	Échec	Instruction obligatoire
`READ(x,);`	Échec	ID requis après une virgule
`WRITE();`	Échec	Expression obligatoire
`x := 10 + ;`	Échec	Primaire requis après un opérateur
`WRITE((x+2);`	Échec	Parenthèse fermante requise
Instruction après END	Échec	SCANEOF obligatoire
`x := -2.5;`	Échec	Signe unaire absent de la grammaire
`x := 1e3;`	Échec	Exposant non reconnu comme partie du nombre
`WRITE(1@);`	Échec lexical	Caractère inconnu ; arrêt immédiat

Ces fragments sont testés à l'intérieur de `BEGIN ... END`, sauf lorsque la structure complète du programme est précisément le point à vérifier.

Cours : pages 20–21, conformité grammaticale ; **pages 40 et 107**, modèles lexicaux et caractères qui ne correspondent à aucun modèle.

Exemple de déroulement complet

Pour `BEGIN x := (2 + 3.5); END`, les appels suivent cette structure :

```
main : lire le premier token BEGIN
```

```
└─ system_goal
   ├─ program
   │  ├─ match BEGIN
   │  ├─ inst_list
   │  │  └─ inst : branche ID
   │  │     ├─ match ID puis ASSIGNOP
   │  │     ├─ expression
   │  │     │  └─ prim : branche LPAREN
   │  │     │     ├─ match LPAREN
   │  │     │     ├─ expression : entier 2, PLUSOP, réel 3.5
   │  │     │     └─ match RPAREN
   │  │     └─ match SEMICOLON
   │  └─ match END
   └─ match SCANEOF
```

Chaque `match` réussi avance vers le token suivant. À chaque retour de fonction, le token courant est celui qui suit le constituant reconnu. Si tout le fichier est consommé selon ces règles, le programme annonce la réussite.

Cours : pages 20–21, structure hiérarchique ; **pages 38 et 100**, échanges successifs entre les deux analyseurs.

Limites et possibilités d'évolution

Le programme est un reconnaisseur lexical et syntaxique, pas un compilateur complet. Il ne construit ni table des symboles (**cours, page 30**), ni arbre conservé en mémoire (**page 21**), ne vérifie pas les types (**page 22**) et ne génère aucun code (**page 23**).

Ajouter *, /, les signes unaires ou les exposants exigerait de revoir les modèles lexicaux ou les règles grammaticales concernées. Afficher plusieurs erreurs nécessiterait une stratégie explicite de récupération. La récursion utilise la pile d'appels C ; aucune limite explicite de profondeur n'est prévue pour des parenthèses extrêmement imbriquées. Le tampon lexical reste limité à 255 caractères et le scanner ne traite pas les identificateurs Unicode.